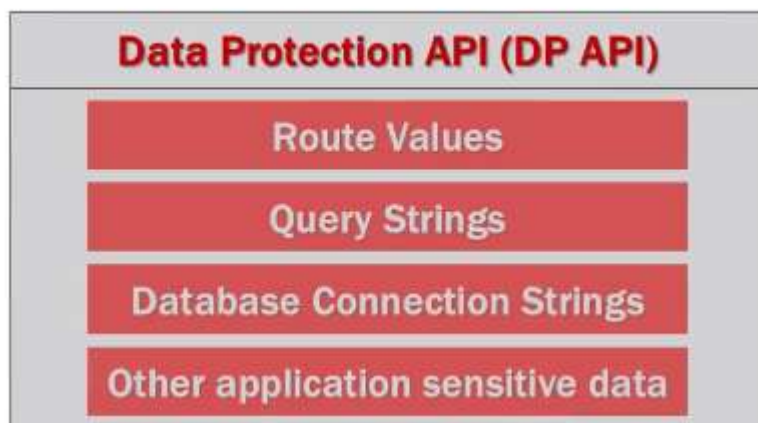


120 - Encryption and Decryption Example

Ще използваме DP API за криптиране и декриптиране на Route value

ASP.NET Core Encryption & Decryption



Като пример, когато натиснем Edit-бутонa, ни прехвърля към /home/edit/2. Ние ще искаме да криптираме ID да не се вижда като integer value 2, за да не може да се прочете.

Security folder -> add -> class DataProtectionPurposeStrings

На този етап той ще съдържа само един purpose string с един property, който наричаме EmployeeIdRouteValue

```
namespace EmployeeManagement.Security
{
    0 references
    public class DataProtectionPurposeStrings
    {
        public readonly string EmployeeIdRouteValue = "EmployeeIdRouteValue";
    }
}
```

Register purpose string class with DI (dependency injection) container:

Това ни позволява да инжектираме instance на този клас във всеки контролер в приложението.

Прелистваме до последно в Startup.cs

```
Startup.cs  X
EmployeeManagement.Startup  Conf

services.AddScoped<IEmployeeRepository, SQLEmployeeRepository>();
services.AddSingleton<IAuthorizationHandler,
    CanEditOnlyOtherAdminRolesAndClaimsHandler>();
services.AddSingleton<IAuthorizationHandler,
    SuperAdminHandler>();
services.AddSingleton<DataProtectionPurposeStrings>();
}
```

```
services.AddSingleton<DataProtectionPurposeStrings>();
```

От 117 урок

<https://github.com/aspnet/Identity/blob/release/2.2/src/Identity/DataProtectionTokenProvider.cs>



```
public class DataProtectorTokenProvider<TUser> : IUserTwoFactorTokenProvider<TUser> where T
{
    /// <summary>
    /// Initializes a new instance of the <see cref="DataProtectorTokenProvider{TUser}" /> class.
    /// </summary>
    /// <param name="dataProtectionProvider">The system data protection provider.</param>
    /// <param name="options">The configured <see cref="DataProtectionTokenProviderOptions" />.</param>
    public DataProtectorTokenProvider(IDataProtectionProvider dataProtectionProvider, IOptions<DataProt
    {
        if (dataProtectionProvider == null)
        {
            throw new ArgumentNullException(nameof(dataProtectionProvider));
        }
        Options = options?.Value ?? new DataProtectionTokenProviderOptions();
        // Use the Name as the purpose which should usually be distinct from others
        Protector = dataProtectionProvider.CreateProtector(Name ?? "DataProtectorTokenProvider");
    }
}
```

Class DataProtectorTokenProvider отговаря за генериране на email confirmation token, password reset token, encryption and decryption tokens, затова и самият клас се нарича така DataProtectorTokenProvider. Като разлистим надолу, той съдържа property Protector:

```
protected IDataProtector Protector { get; private set; }
           type           prop
```

Който съдържа 2 ценни метода: protect и unprotect, които ще използваме.

Надолу виждаме методът, който генерира токен,

```
public virtual async Task<string> GenerateAsync(string purpose, UserManager<TUser> manager, TUser user)
{
```

А генерираният токен после се подава на Protect метод на IDataProtector, така че токентът може да бъде криптиран. По-надолу имаме метод Validate, и Unprotect за декриптиране.

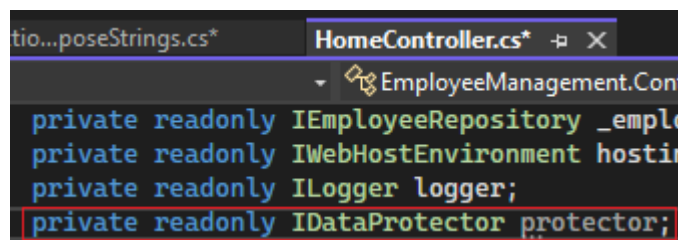
```

    var protectedBytes = Protector.Protect(ms.ToArray());
    return Convert.ToBase64String(protectedBytes);
}

/// <summary>
/// Validates the protected <paramref name="token"/> for the specified <paramref nam
/// </summary>
/// <param name="purpose">The purpose the token was be used for.</param>
/// <param name="token">The token to validate.</param>
/// <param name="manager">The <see cref="UserManager{TUser}"/> to retrieve user prop
/// <param name="user">The <typeparamref name="TUser"/> the token was generated for.
/// <returns>
/// A <see cref="Task{TResult}"/> that represents the result of the asynchronous val
/// containing true if the token is valid, otherwise false.
/// </returns>
public virtual async Task<bool> ValidateAsync(string purpose, string token, UserMana
{
    try
    {
        var unprotectedData = Protector.Unprotect(Convert.FromBase64String(token));
    }
}

```

Ще използваме всичко това, за да криптираме и декриптираме ID в URL-адреса.



```

tio...poseStrings.cs*   HomeController.cs*
- EmployeeManagement.Con
private readonly IEmployeeRepository _emplo
private readonly IWebHostEnvironment hostin
private readonly ILogger logger;
private readonly IDataProtector protector;

```

```
private readonly IDataProtector protector;
```

За да го използваме, нека разберем какъв е принципът.

How to get an instance of the class that implements IDataProtector. How the build-in DataProtector provide a class? Notice, to get an instance of IDataProtector, we are using DataProtectionProvider and this variable is of type IDataProtectionProvider. This interface is injected into the class, using constructor and then on this interface we have CreateProtector method, and this method is passing the purpose string. They are using the Name of the DataProtectorTokenProvider to use as a purpose string. In our case, we have a separate class, that contains our purpose string. So in addition to injecting this interface IDataProtectionTokenProvider, we also need to inject our class that contains our purpose string. Let's first inject IDataProtectionProvider into our HomeController.

```

io...poseStrings.cs* HomeController.cs*
EmployeeManagement.Controllers.HomeController HomeController(IEm
private readonly IWebHostEnvironment hostingEnvironment; //auto private field
private readonly ILogger logger;
private readonly IDataProtector protector;

0 references
public HomeController(IEmployeeRepository employeeRepository,
    IWebHostEnvironment hostingEnvironment,
    ILogger<HomeController> logger,
    IDataProtectionProvider dataProtectionProvider,
    DataProtectionPurposeStrings dataProtectionPurposeStrings)
{
    _employeeRepository = employeeRepository;
    this.hostingEnvironment = hostingEnvironment;
    this.logger = logger;
    protector = dataProtectionProvider
        .CreateProtector(dataProtectionPurposeStrings.EmployeeIdRouteValue);
}

```

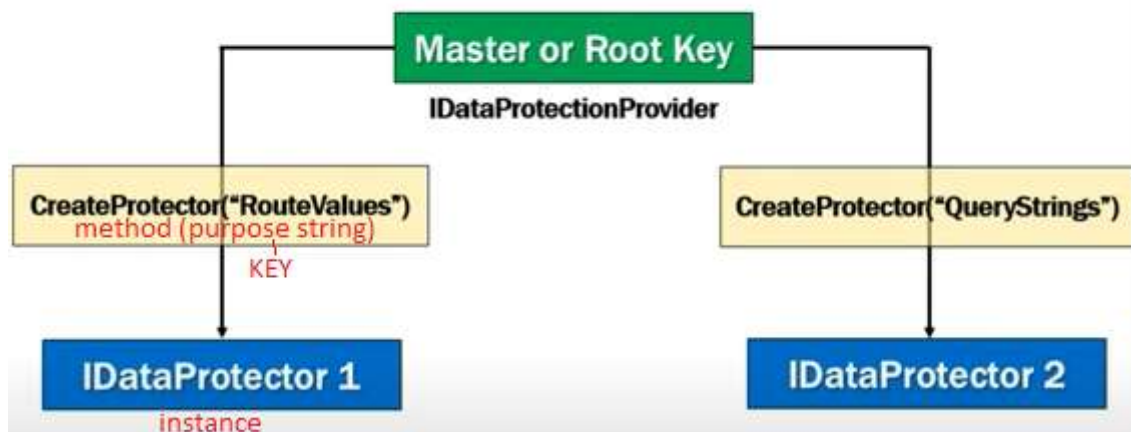
```

private readonly IDataProtector protector;

public HomeController(IEmployeeRepository employeeRepository,
    IWebHostEnvironment hostingEnvironment,
    ILogger<HomeController> logger,
    IDataProtectionProvider dataProtectionProvider,
    DataProtectionPurposeStrings
dataProtectionPurposeStrings)
{
    _employeeRepository = employeeRepository;
    this.hostingEnvironment = hostingEnvironment;
    this.logger = logger;
    protector = dataProtectionProvider
        .CreateProtector(dataProtectionPurposeStrings.EmployeeIdRouteValue);
}

```

ASP.NET Core Encryption & Decryption



You can think of purpose string as an encryption key. This key is then combined with the master or root key to generate a unique key. The data that is encrypted by a given combination of purpose string and root key can only be decrypted by that same combination of keys.

The purpose string is inherent to the security of the data protection system, as it provides isolation between cryptographic consumers, even if the root keys are the same.

Ако погледнем Employee модела, виждаме, че има property, което съхранява четимия текст. Необходимо ни е още едно property за криптирания текст и за да не бъде асоцииран с нито една колона от БД, слагаме атрибут [NotMapped]

```
namespace EmployeeManagement.Models
{
    45 references
    public class Employee
    {
        17 references
        public int Id { get; set; }
        [NotMapped]
        0 references
        public string EncryptedId { get; set; }
        [Required]
        [MaxLength(60, ErrorMessage = "Name cannot exceed 60 characters")]
    }
}
```

```
[NotMapped]
public string EncryptedId { get; set; }
```

Това криптирано ID трябва да важи и в index страницата, където се намира списъкът с Employees.

```
Employee.cs
HomeController.cs*
EmployeeManagement.Controllers.HomeCo

[AllowAnonymous]
0 references
public IActionResult Index()
{
    var model = _employeeRepository.GetAllEmployee();
    return View(model);
}
```

We use the link Select statement,

e.EncryptedId присвоява e.ID plaintext и го конвертира ToString.

За да се осъществи самото криптиране, използваме protector, който създадохме в конструктора и неговият .Protect метод.

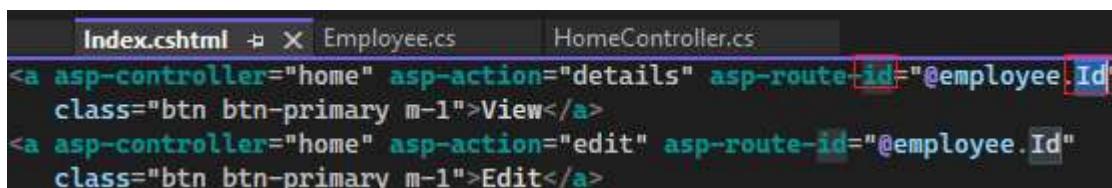
```
[AllowAnonymous]
0 references
public IActionResult Index()
{
    var model = _employeeRepository.GetAllEmployee()
        .Select(e =>
        {
            e.EncryptedId = protector.Protect(e.Id.ToString());
            return e;
        });
    return View(model);
}
```

```

[AllowAnonymous]
public IActionResult Index()
{
    var model = _employeeRepository.GetAllEmployee()
        .Select(e =>
        {
            // Encrypt the ID value and store in EncryptedId property
            e.EncryptedId = protector.Protect(e.Id.ToString());
            return e;
        });
    return View(model);
}

```

Имаме 1 грешка, Нека погледнем Index view – маркира Index action, защото EncryptedID все още не се използва.

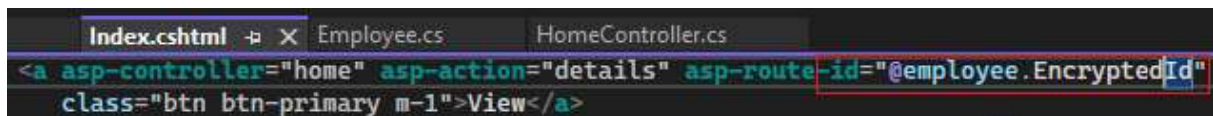


```

<a asp-controller="home" asp-action="details" asp-route-id="@employee.Id"
class="btn btn-primary m-1">View</a>
<a asp-controller="home" asp-action="edit" asp-route-id="@employee.Id"
class="btn btn-primary m-1">Edit</a>

```

Вместо видимото Id, нека сложим EncryptedID и видим резултата.



```

<a asp-controller="home" asp-action="details" asp-route-id="@employee.EncryptedId"
class="btn btn-primary m-1">View</a>

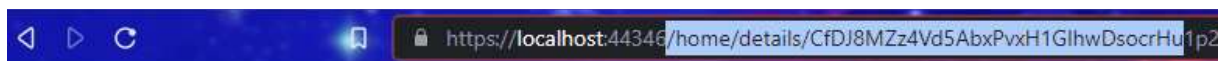
```

```

<a asp-controller="home" asp-action="details" asp-route-id="@employee.EncryptedId"

```

Когато изберем бутон view от списъка, получаваме грешка.



An unhandled exception occurred while processing the request.

InvalidOperationException: Nullable object must have a value.

System.Nullable<T>.get_Value()

Stack

Query

Cookies

Headers

Routing

InvalidOperationException: Nullable object must have a value.

System.Nullable<T>.get_Value()

EmployeeManagement.Controllers.HomeController.Details(Nullable<int> id) in HomeController.cs

59. Employee employee = _employeeRepository.GetEmployee(id.Value);

lambda_method210(Closure, object, object[])

Линкът, вместо откритото ID, съдържа криптираното.

HomeController.Details(Nullable<int> id

```
tion...poseStrings.cs | HomeController.cs | Ind
EmployeeManagement.Co
0 references
public IActionResult Details(int? id)
{
    new encrypted id is of type string
}
```

Сменяме типа на Id да съответства на string, както е при криптираното Id,

След това създаваме нова променлива, тип стринг

```
string decryptedId = protector.Unprotect(id);
Employee employee = _employeeRepository.GetEmployee(id.Value);
if (employee == null)
{
    Response.StatusCode = 404;
    return View("EmployeeNotFound", id.Value);
}
```

но понеже не я използваме никъде другаде, можем да сменим името ѝ, за да е по-ясно, също типът да бъде int, за да може да се използва долу, като имаме предвид, че метод Unprotect връща резултат string, значи използваме .ToInt32 за да го конвертираме от string -> int и използваме по този начин.

```
//string decryptedId = protector.Unprotect(id);
int employeeId = Convert.ToInt32(protector.Unprotect(id));
Employee employee = _employeeRepository.GetEmployee(employeeId);
if (employee == null)
{
    Response.StatusCode = 404;
    return View("EmployeeNotFound", employeeId);
}
```

/home/details/CfDJ8MZz4Vd5AbxPvxH1GlhwDsqx0jy4jRhB1KSR8-GIAvH95

encrypted ID

John



Employee ID: 2

ID в адреса на брауъра е криптирано, но вътре като информация в страницата е видимо.